ARCHITECTURE & SECURITY REFERENCE DOCUMENTATION

SPIFFE/SPIRE + OAuth 2.0 (RFC 8693) + MCP Security Sandbox

Detailed component specification and runtime workflow design for `spiffe-poc`: proving zero-trust workload attestation, coroutine-level human session isolation inside an AI agent tool harness, on-behalf-of (OBO) token exchange, and double-locked intersection authorization at an MCP server.

Trust Domain: example.org | **SPIRE:** 1.10.3 (Server & Agent) | **OAuth Flow:** RFC 8693 Token Exchange | **Runtime:** Python 3.12 + FastAPI + spiffe 0.2.3 | **Containers:** 6 Docker Compose Services

1. Executive Overview & Architectural Guardrails

Autonomous AI agents execute backend tool calls over shared runtimes serving multiple concurrent users. Putting user credentials into LLM prompts risks prompt-injection leakage, while raw OAuth bearer tokens can be replayed by unrelated workloads unless bound to the caller's attested workload identity. This sandbox enforces four strict guardrails:

1. Zero LLM Credential Access

Human credentials (`subject_token`) never enter the simulated LLM layer and are never passed as tool parameters. Sessions are isolated per async coroutine via Python's `contextvars.ContextVar` (`current_user_token`).

2. Environment-Driven SPIFFE Socket

Every workload container sets `SPIFFE_ENDPOINT_SOCKET=unix:///spire-socket/api.sock` and calls `WorkloadApiClient()` with zero hardcoded paths, ensuring portable, secret-free workload identity.

3. Host PID & Unix UID Attestation

`spire-agent` shares the host PID namespace (`pid: "host"`) and loads the `unix` WorkloadAttestor. Workloads run under dedicated UIDs (`1001` – `1004`) for reliable attestation across Linux, macOS Docker Desktop, and WSL2.

4. Double-Lock + Intersection AuthZ

The MCP Server enforces four sequential gates: live `X-SPIFFE-SVID` validation, OBO JWT signature and audience check, actor binding (`live_caller_spiffe_id == act.sub`), and intersection permissions for both bot and human roles.

2. Identity & Service Topology Matrix

Service / File	Linux UID	SPIFFE ID (`example.org`)	Interface / Port	Primary Role
`spire-server` `server.conf`	root	Control Plane Root CA	TCP `:8081` Admin UDS	Trust domain root CA, SQLite entry datastore, join-token node attestation, X.509/JWT-SVID signing.
`spire-agent` `agent.conf`	root (`pid:host`)	`spiffe://example.org/host`	`/spire-socket/api.sock`	Node attestation via join token; Unix UID workload attestation; local SPIFFE Workload & SDS API.
`auth-server` `auth_server.py`	`1001` (`authuser`)	`spiffe://example.org/ns/security/auth-server`	HTTP `:8001` `/oauth/token`	RFC 8693 Token Exchange STS; validates human token + agent JWT-SVID and mints OBO Combo Pass JWT.
`mcp-server` `mcp_server.py`	`1002` (`mcpuser`)	`spiffe://example.org/ns/mcp/billing-server`	HTTP `:8002` `/mcp/tools/call`	Policy Enforcement Point (PEP) & billing resource server; runs 4-gate checks and serves `INVOICE_DB` .
`ai-agent` `agent.py`	`1003` (`agentuser`)	`spiffe://example.org/ns/ai/finops-bot`	Client (HTTP + UDS)	Legitimate AI workload hosting isolated user coroutines, LLM intent boundary, and security tool harness.
`rogue-agent` `attack_test.py`	`1004` (`rogueuser`)	`spiffe://example.org/ns/ai/rogue-bot`	Client (HTTP + UDS)	Adversarial workload verifying that replaying Alice's stolen OBO JWT fails Gate 3 Actor Binding.

3. End-to-End Architecture & Workflow Diagram

Figure 1 depicts the multi-container topology across the **SPIFFE/SPIRE Identity Control Plane** (top), the shared `/spire-socket/api.sock` Workload API bus, and the three runtime flows: bootstrap attestation (`B1–B2`), concurrent sessions for Alice & Bob (`1–7`), and stolen-token replay defense (`A1–A3`).

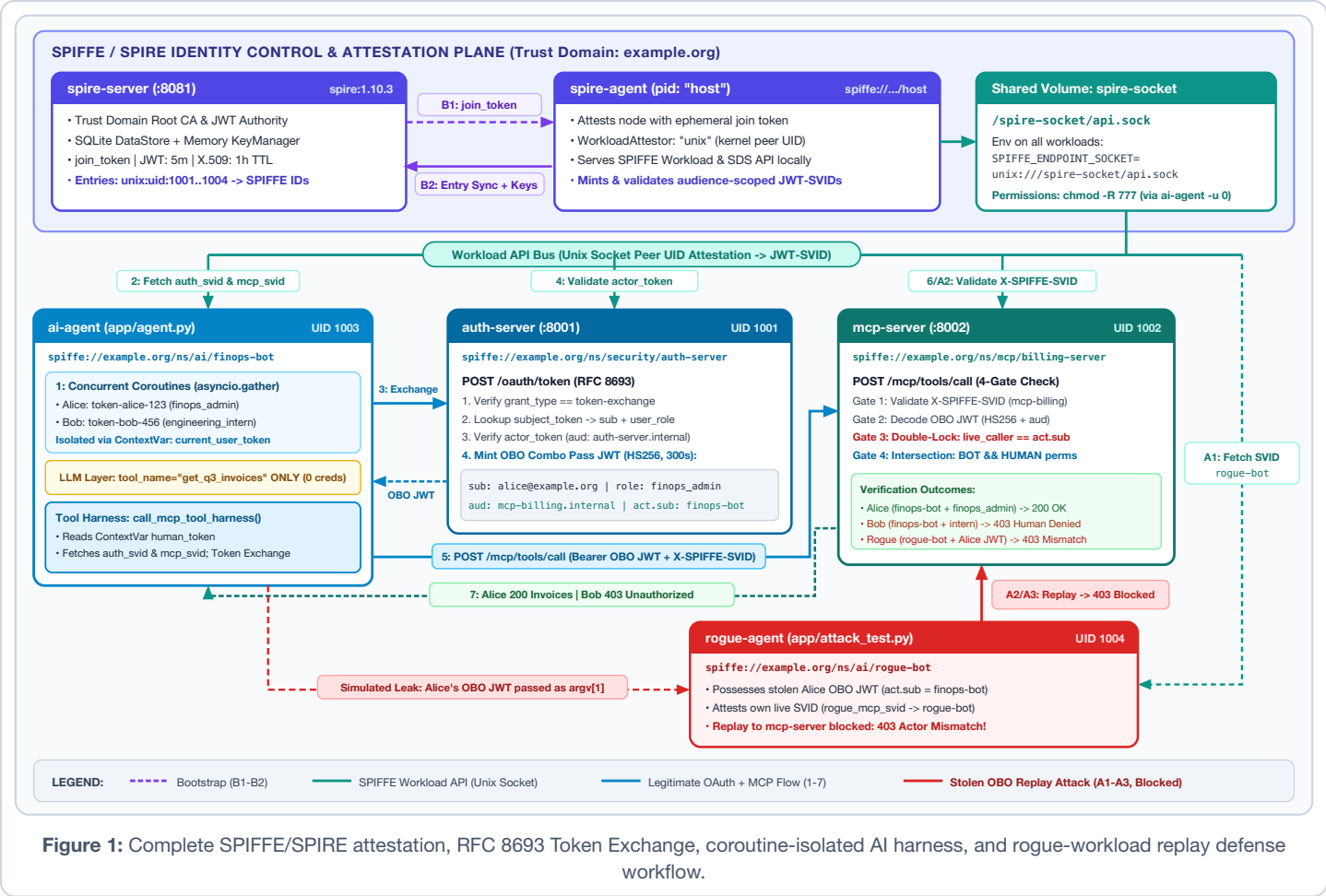


Figure 1: Complete SPIFFE/SPIRE attestation, RFC 8693 Token Exchange, coroutine-isolated AI harness, and rogue-workload replay defense workflow.

End-to-End Runtime Workflow Sequence

1. **Bootstrap (`B1–B2`):** `bootstrap.sh` requests a node join token from `spire-server` , boots `spire-agent` (`pid: "host"`), exposes `/spire-socket` to UIDs `1001–1004` , and registers four SPIFFE entries under parent `spiffe://example.org/host` .
2. **Session Isolation (`1`):** `agent.py` runs Alice (`finops_admin`) and Bob (`engineering_intern`) concurrently via `asyncio.gather()` , binding each user's token into an isolated `contextvars.ContextVar` before crossing the credential-free LLM boundary.
3. **Workload Attestation (`2`):** `call_mcp_tool_harness()` uses `WorkloadApiClient()` to fetch two audience-scoped JWT-SVIDs (`auth_svid` , `mcp_svid`); SPIRE Agent verifies kernel peer UID `1003` and attests `finops-bot` .
4. **Token Exchange (`3–4`):** The harness posts `subject_token` + `actor_token` (`auth_svid`) to `auth-server:8001/oauth/token` . `auth-server` validates `auth_svid` via SPIRE and issues an OBO JWT binding human `sub` / `user_role` with `act.sub = finops-bot` .
5. **Double-Lock Enforcement (`5–7`):** The harness calls `mcp-server:8002/mcp/tools/call` with `Authorization: Bearer <OBO_JWT>` and `X-SPIFFE-SVID: <mcp_svid>` . `mcp-server` validates the SVID, verifies the OBO JWT, and checks intersection permissions (Alice `200 OK` , Bob `403`).
6. **Replay Defense (`A1–A3`):** `rogue-agent` (UID `1004`) steals Alice's OBO JWT and presents its own SVID (`rogue-bot`); `mcp-server` rejects the call at Gate 3 with `403 Actor mismatch!` .

4. Detailed Component Responsibilities & Functions

4.1 SPIRE Server (Control Plane & Trust Root)

`spire/server/server.conf | spiffe/spire-server:1.10.3`

Responsibility: Central identity authority and SVID issuer for trust domain `example.org` on TCP port `8081`.

- **Trust Root & Signing TTLs:** Uses `KeyManager "memory"` with `default_x509_svid_ttl = "1h"` and `default_jwt_svid_ttl = "5m"`.
- **Registration Datastore:** Persists node and workload entries in SQLite3 (`/opt/spire/data/server/datastore.sqlite3` via `DataStore "sql"`).
- **Node Attestation & Entry CLI:** Uses `NodeAttestor "join_token"` so `bootstrap.sh` can generate one-time join tokens (`spire-server token generate`) and register UID selectors (`spire-server entry create ... -selector unix:uid:...`).

4.2 SPIRE Agent (Node Attestor & Workload API Daemon)

`spire/agent/agent.conf | pid: "host" | /spire-socket/api.sock`

Responsibility: Host-level identity proxy attesting local container processes and serving the SPIFFE Workload API without placing static credentials inside workload containers.

- **Node Bootstrapping:** Connects to `spire-server:8081` using `-joinToken` and `insecure_bootstrap = true`, attaining `spiffe://example.org/host`.
- **Kernel Peer Inspection (`WorkloadAttestor "unix"`):** Sharing the host PID namespace (`pid: "host"`) allows SPIRE Agent to inspect the kernel UID of any process connecting over `/spire-socket/api.sock` across Docker Desktop/Linux/WSL2.
- **Local Workload API:** Caches bundles/SVIDs, handles rotation automatically, and serves `fetch_jwt_svid` and `validate_jwt_svid` over gRPC/UDS.

4.3 SVIDs (SPIFFE Verifiable Identity Documents — JWT-SVIDs)

`spiffe==0.2.3 | WorkloadApiClient() | 5m Default TTL`

Responsibility: Short-lived cryptographic workload badges proving *which OS workload* is invoking an API, strictly scoped to an audience URI.

- **Audience-Scoped Issuance:** `agent.py` fetches two distinct JWT-SVIDs— `auth_svid` (`audience={"https://auth-server.internal"}`) and `mcp_svid` (`audience={"https://mcp-billing.internal"}`)—blocking cross-service SVID reuse.
- **Workload API Validation:** Both `auth_server.py` and `mcp_server.py` call `spiffe_client.validate_jwt_svid(token=..., audience=...)` so SPIRE Agent performs cryptographic verification against live trust-domain keys.
- **Socket Auto-Discovery:** `WorkloadApiClient()` reads `SPIFFE_ENDPOINT_SOCKET=unix:///spire-socket/api.sock` directly from container environment variables.

4.4 AI Agent & Tool-Call Harness (Session Isolation Layer)

`app/agent.py | ai-agent | UID 1003 | finops-bot`

Responsibility: Executes concurrent user sessions while ensuring the simulated LLM layer never touches human or SPIFFE credentials.

- **`current_user_token` (`contextvars.ContextVar[str]`):** Isolates each async coroutine's human token (`token-alice-123` vs `token-bob-456`) across concurrent `asyncio.gather()` execution.
- **`simulate_user_request(label, login_token)`:** Sets the `ContextVar`, calls the tool harness passing only `tool_name="get_q3_invoices"`, and resets the token reference in a `finally` block.
- **`call_mcp_tool_harness(tool_name)` (The Harness):** Reads `current_user_token.get()`, fetches `auth_svid` & `mcp_svid`, performs RFC 8693 token exchange with `auth-server`, and calls `mcp-server` with `Authorization` + `X-SPIFFE-SVID` headers.
- **`main()`:** Runs Alice and Bob concurrently, asserting HTTP `200` for Alice and HTTP `403` for Bob.

4.5 OAuth 2.0 RFC 8693 Token Exchange Server

`app/auth_server.py | auth-server (:8001) | UID 1001`

Responsibility: Security Token Service (STS) exchanging a human `subject_token` + verified agent `actor_token` into an On-Behalf-Of (OBO) Combo Pass JWT.

- **Endpoints:** `GET /health` (`{"status": "ok"}`) and `POST /oauth/token` -> `token_exchange(grant_type, subject_token, actor_token, resource, scope)` .
- **Verification:** Enforces `grant_type == "urn:ietf:params:oauth:grant-type:token-exchange"` , resolves human identity/role from `USER_TOKENS` , and validates `actor_token` against SPIRE (`audience="https://auth-server.internal"`).
- **OBO JWT Minting:** Signs a 300s HS256 JWT (`OAUTH_SIGNING_SECRET`) embedding `iss` , `sub` , `user_role` , `aud=resource` , `scope` , `iat` , `exp` , and `act: {"sub": agent_spiffe_id}` .

4.6 MCP Billing Server (Double-Lock Policy Enforcement Point)

`app/mcp_server.py | mcp-server (:8002) | UID 1002`

Responsibility: Exposes the protected `get_q3_invoices` tool (`POST /mcp/tools/call` , schema `MCPToolRequest`) and enforces 4-gate verification before querying `INVOICE_DB` .

- **Gate 1 (Live SPIFFE):** Validates `X-SPIFFE-SVID` header via SPIRE (`audience="https://mcp-billing.internal"`) -> `live_caller_spiffe_id` .
- **Gate 2 (OBO JWT):** Decodes `Authorization: Bearer <OBO_JWT>` verifying HS256 signature, expiration, and `aud` .
- **Gate 3 (Double-Lock):** Rejects if `live_caller_spiffe_id != claims["act"]["sub"]` (HTTP `403 Actor mismatch!`).
- **Gate 4 (Intersection AuthZ):** Requires `tool_name` in both `BOT_PERMISSIONS[live_caller_spiffe_id]` and `HUMAN_PERMISSIONS[user_role]` ; returns invoices scoped strictly to `claims["sub"]` .

4.7 Rogue Agent Replay Test, Container Build & Bootstrap Automation

`app/attack_test.py | Dockerfile | docker-compose.yml | bootstrap.sh`

- **`attack_test.py` (`rogue-agent` , UID `1004`):** Attests as `spiffe://example.org/ns/ai/rogue-bot` , fetches `rogue_mcp_svid` , replays Alice's stolen OBO JWT (`sys.argv[1]`), and asserts HTTP `403` with `"Actor mismatch"` .
- **`app/Dockerfile` & `requirements.txt` :** Python `3.12-slim` image with FastAPI `0.115.0` , Uvicorn `0.30.6` , HTTPX `0.27.2` , PyJWT `2.9.0` , `spiffe==0.2.3` , `python-multipart==0.0.9` , and Linux UIDs `1001-1004` .
- **`bootstrap.sh` & `docker-compose.yml` :** 6-stage lifecycle orchestration: (1) teardown prior containers/volumes, (2) build & start `spire-server` , (3) mint join token for `spiffe://example.org/host` , (4) start `spire-agent` + workloads and run `docker compose exec -T -u 0 ai-agent chmod -R 777 /spire-socket` (because `spire-agent:1.10.3` is distroless), (5) register `unix:uid:1001..1004` entries, (6) sleep 6s and run `agent.py` + `attack_test.py` .

4.8 Configuration & Workspace Artifact Reference

File Path	Key Settings / Dependencies	Purpose in Sandbox
`spire/server/server.conf`	`trust_domain = "example.org"` , `:8081` , `sqlite3`	Configures SPIRE Server control plane, 1h X.509 / 5m JWT TTLs, and `join_token` node attestor.
`spire/agent/agent.conf`	`socket_path = "/spire-socket/api.sock"` , `unix`	Connects agent to `spire-server:8081` and enables Unix kernel UID workload attestation.
`docker-compose.yml`	`pid: "host"` , volume `spire-socket` , UIDs `1001-1004`	Wires the 6 containers, shared UDS mount, and `SPIFFE_ENDPOINT_SOCKET` environment variable.
`app/Dockerfile` & `requirements.txt`	`python:3.12-slim` , `fastapi==0.115.0` , `spiffe==0.2.3` , `PyJWT==2.9.0`	Creates isolated non-root Linux users (`authuser` , `mcpuser` , `agentuser` , `rogueuser`) and installs runtime libs.
`bootstrap.sh`	`spire-server token generate` & `entry create`	Automates build, join-token exchange, socket `chmod` , entry registration, and both verification tests.

5. Token Anatomy & MCP 4-Gate Double-Lock Pipeline

Figure 2 illustrates the cryptographic link between the two headers presented to `mcp-server` and the four sequential verification gates inside `call_tool()`.

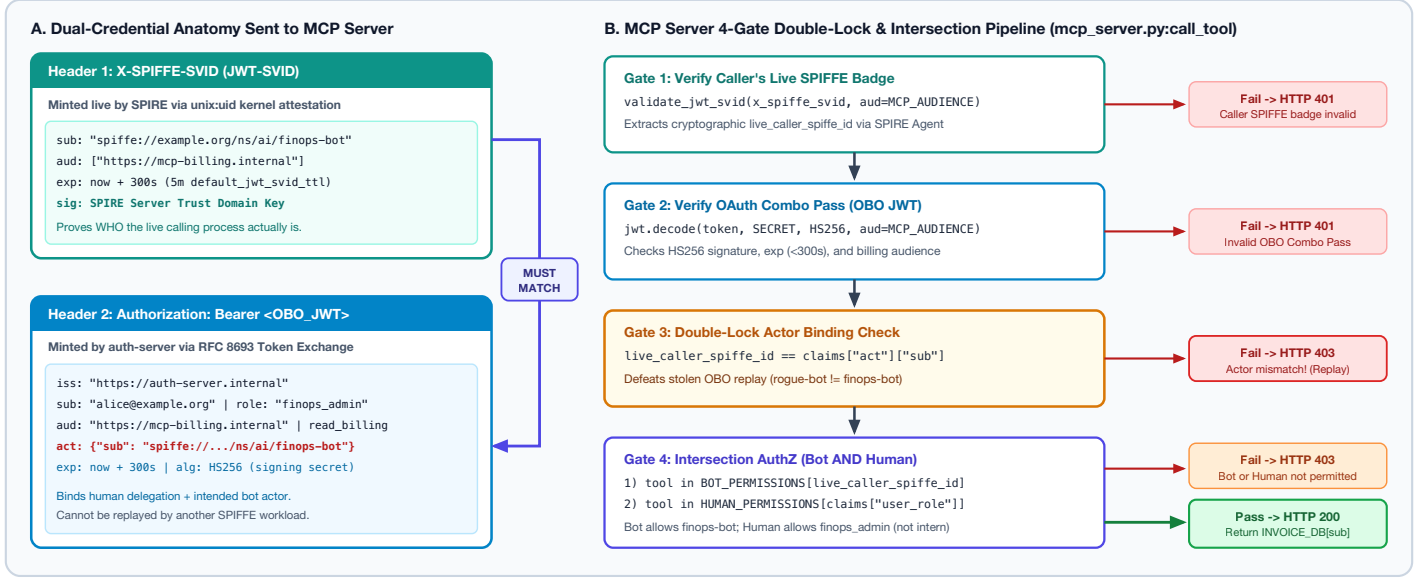


Figure 2: Dual-credential claims structure and the 4-gate Double-Lock + Intersection AuthZ pipeline in `mcp_server.py`.

6. Threat Mitigation & Verified Execution Matrix

Security Threat / Scenario	Enforcing Mechanism & Code Path	Verified Runtime Outcome (from `./bootstrap.sh`)
Legitimate Admin Delegation (Alice -> `finops-bot`)	`ContextVar` + RFC 8693 OBO + Double-Lock + Intersection AuthZ	**HTTP 200 OK:** Returns `alice@example.org` invoices (`$4,200 AWS`, `$1,150 GCP`) with `executed_by_bot: finops-bot`.
Human Privilege Escalation (Bob -> `finops-bot`)	Gate 4 Check: `HUMAN_PERMISSIONS["engineering_intern"] == []`	**HTTP 403 Forbidden:** `Human user 'bob@example.org' (role: engineering_intern) is not authorized for 'get_q3_invoices'`.
Stolen OBO Token Replay (`rogue-bot` using Alice's JWT)	Gate 3 Double-Lock: `live_caller_spiffe_id != claims["act"]["sub"]`	**HTTP 403 Forbidden:** `Actor mismatch! Live caller is '.../rogue-bot', but token is bound to '.../finops-bot'`.
Concurrent Coroutine Leakage (Alice & Bob in parallel)	`current_user_token: ContextVar[str]` set/reset per coroutine	**Isolated:** Concurrent `asyncio.gather()` sessions resolve independent tokens and AuthZ results with zero cross-talk.
Cross-Service SVID Reuse (`auth_svid` sent to MCP)	Audience check in `validate_jwt_svid(..., audience=MCP_AUDIENCE)`	**HTTP 401 Unauthorized:** SVID minted for `https://auth-server.internal` is rejected at `https://mcp-billing.internal`.
Container UID Impersonation	SPIRE `WorkloadAttestor "unix"` + `user: "1001..1004"` + `pid: "host"`	**Blocked by OS/SPIRE:** Each workload only receives SVIDs matching its kernel-verified UID registration entry.

Production Hardening Note: This sandbox intentionally uses `insecure_bootstrap = true` on `spire-agent`, `KeyManager "memory"`, symmetric `HS256` with a local lab secret (`OAUTH_SIGNING_SECRET`), and static mock `USER_TOKENS` for self-contained local testing. Production deployments should substitute attested node bootstrapping (e.g., cloud/k8s_psat), asymmetric RS256/ES256 OIDC JWKS validation, external IdP subject tokens, and X.509-SVID mTLS.